Student Name	K.VYSHNAVI	
Student Registration Number	232P4R1035	**Class & Section: MCA-A(Regular)**
Study Level: UG/PG	PG	**Year & Term: 2024 &2nd year 3rd TERM**
Subject Name	DEEP LEARNING	

Name of the Assessment	**LAB REPORT-9**
Date of Submission	04/07/2025

LAB REPORT-9

1.

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences

# 1. Load IMDB dataset (top 10,000 words)
num_words = 10000
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=num_words)

# 2. Pad sequences to have equal length (max 200 words)
maxlen = 200
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

# 3. Build the model
model = models.Sequential([
    layers.Embedding(input_dim=num_words, output_dim=128, input_length=maxlen),
    layers.LSTM(64, return_sequences=False),
    layers.Dense(1, activation='sigmoid')
])
```

```
# 2. Pad sequences to have equal length (max 200 words)
maxlen = 200
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

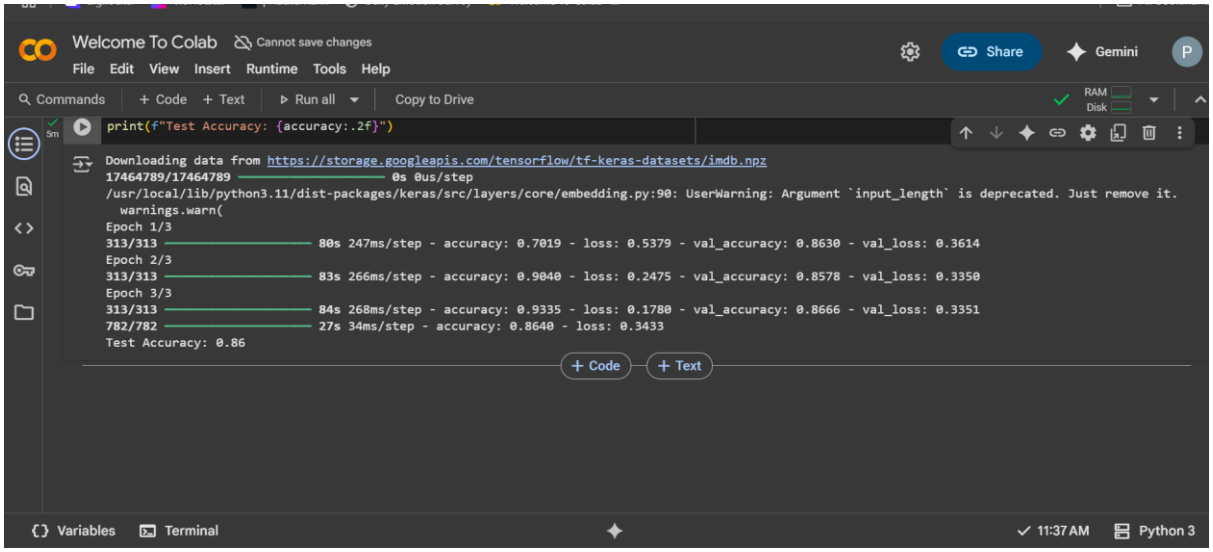
# 3. Build the model
model = models.Sequential([
    layers.Embedding(input_dim=num_words, output_dim=128, input_length=maxlen),
    layers.LSTM(64, return_sequences=False),
    layers.Dense(1, activation='sigmoid')
])

# 4. Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 5. Train the model
model.fit(x_train, y_train, epochs=3, batch_size=64, validation_split=0.2)

# 6. Evaluate on test set
loss, accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {accuracy:.2f}")
```

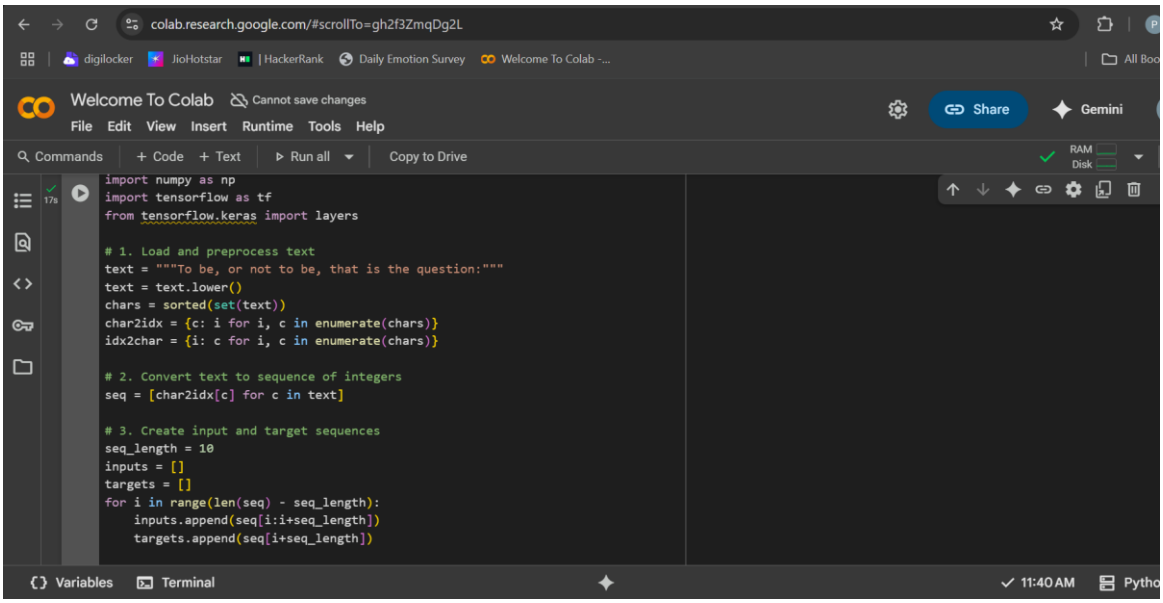
Output:

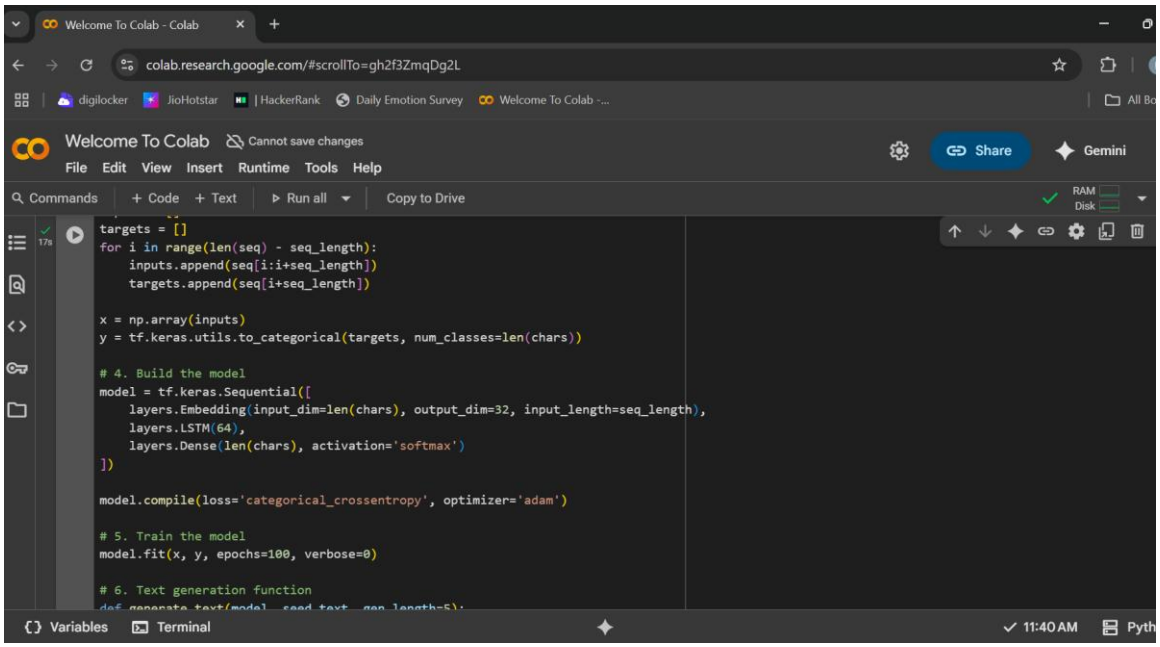


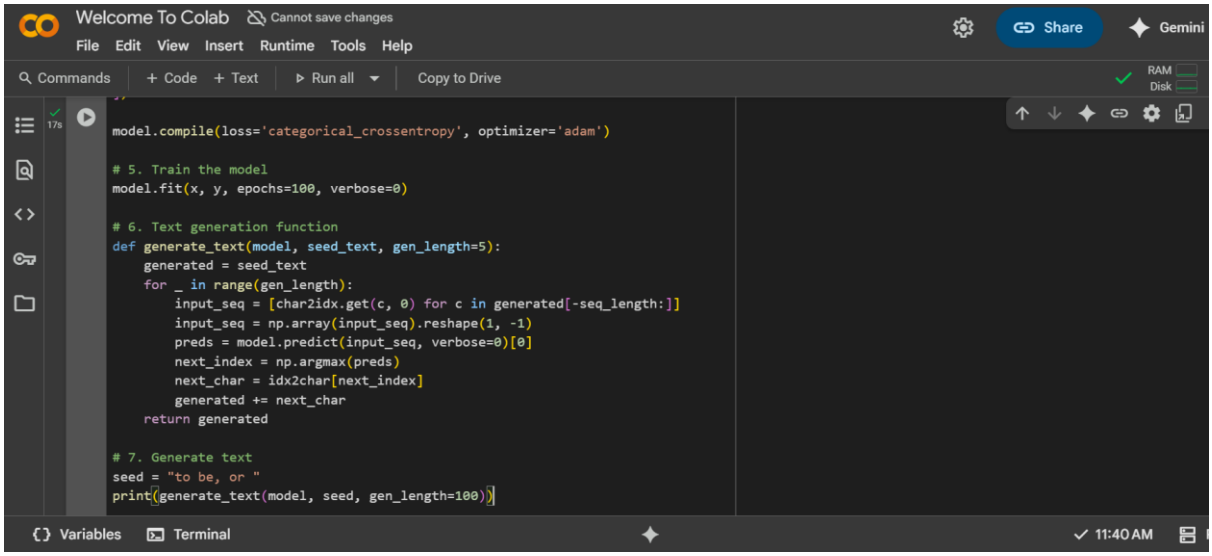
Explanation:

This program trains a model to tell if a movie review is positive or negative using the IMDB dataset. It first prepares the data by keeping only the top 10,000 words and making all reviews the same length. Then, it builds a model with an embedding layer, an LSTM layer to understand the sequence of words, and a final layer to predict sentiment. It trains the model for 3 rounds and then tests its accuracy on new data.

2.




Welcome To Colab - Colab
colab.research.google.com/#scrollTo=gh2f3ZmqDg2L
Welcome To Colab
Cannot save changes
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
Copy to Drive
RAM
Disk
targets = []
for i in range(len(seq) - seq_length):
inputs.append(seq[i:i+seq_length])
targets.append(seq[i+seq_length])
x = np.array(inputs)
y = tf.keras.utils.to_categorical(targets, num_classes=len(chars))
4. Build the model
model = tf.keras.Sequential([
layers.Embedding(input_dim=len(chars), output_dim=32, input_length=seq_length),
layers.LSTM(64),
layers.Dense(len(chars), activation='softmax')
])
model.compile(loss='categorical_crossentropy', optimizer='adam')
5. Train the model
model.fit(x, y, epochs=100, verbose=0)
6. Text generation function
Variables
Terminal
11:40 AM

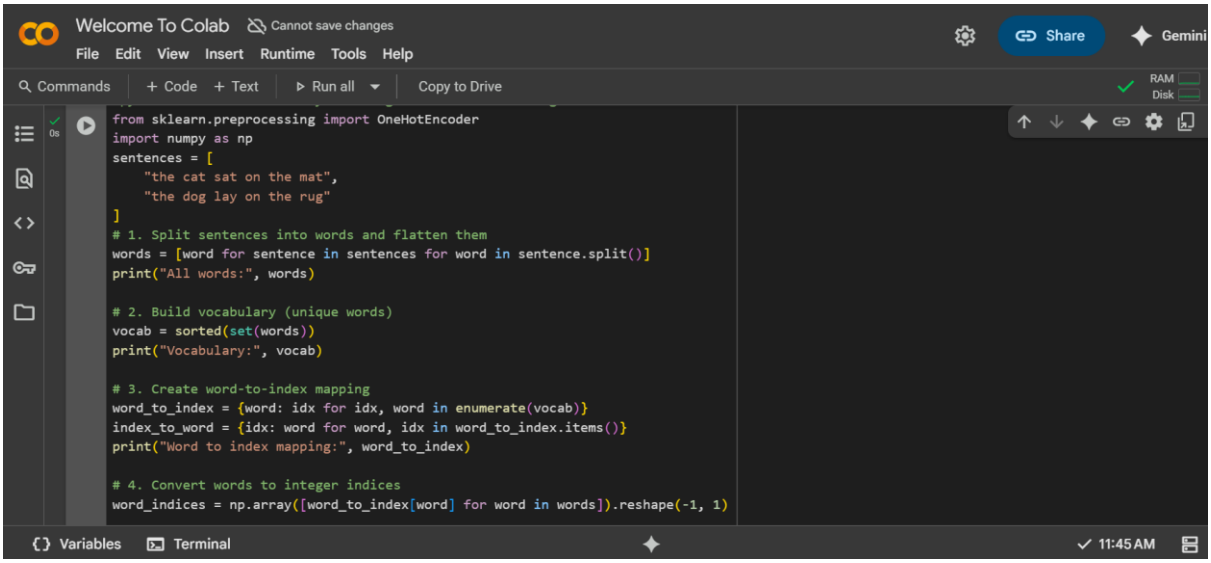

Welcome To Colab
Cannot save changes
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
Copy to Drive
RAM
Disk
model.compile(loss='categorical_crossentropy', optimizer='adam')
5. Train the model
model.fit(x, y, epochs=100, verbose=0)
6. Text generation function
def generate_text(model, seed_text, gen_length=5):
generated = seed_text
for _ in range(gen_length):
input_seq = [char2idx.get(c, 0) for c in generated[-seq_length:]]
input_seq = np.array(input_seq).reshape(1, -1)
preds = model.predict(input_seq, verbose=0)[0]
next_index = np.argmax(preds)
next_char = idx2char[next_index]
generated += next_char
return generated
7. Generate text
seed = "to be, or "
print(generate_text(model, seed, gen_length=100))
Variables
Terminal
11:40 AM

Output:

```
to be, or tt  to  eeeett tin::equuuuu::::::::eeethtonnnn  e e,t t iin   eeeeuttinn:::quuuues::::::::o eet  to
```

Explanation: this program generates text one character at a time using a simple LSTM model. It starts with a short piece of text ("To be, or not to be, that is the question:") and first converts it to lowercase and into a set of unique characters. Each character is then assigned a unique number to turn the text into a sequence of integers. The program creates training data by forming input sequences of 10 characters and setting the following character as the target. It builds a neural network with an embedding layer to represent characters as vectors, an LSTM layer to learn patterns in the sequence, and a dense output layer with softmax to predict the next character. After training the model for 100 epochs, it can generate new text by predicting one character at a time, starting from a given seed like "to be, or ". Since the original text is very short, the output may often contain repeated characters, but it demonstrates the basic idea of how character-level text generation works.

3.



```
# 2. Build vocabulary (unique words)
vocab = sorted(set(words))
print("Vocabulary:", vocab)

# 3. Create word-to-index mapping
word_to_index = {word: idx for idx, word in enumerate(vocab)}
index_to_word = {idx: word for word, idx in word_to_index.items()}
print("Word to index mapping:", word_to_index)

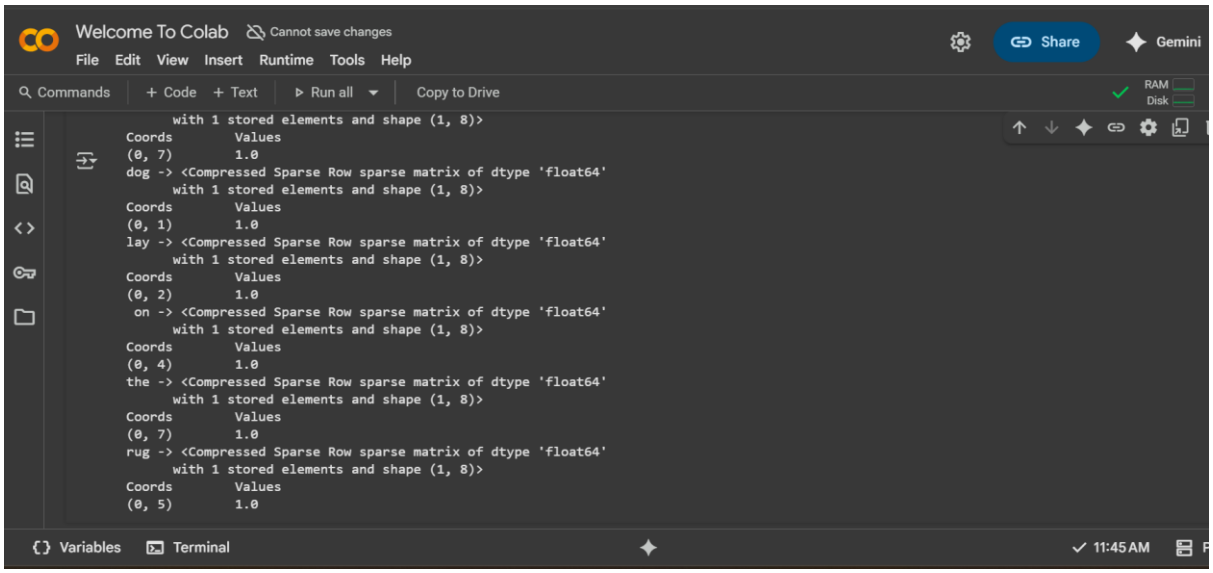
# 4. Convert words to integer indices
word_indices = np.array([word_to_index[word] for word in words]).reshape(-1, 1)

# 5. Apply One-Hot Encoding
encoder = OneHotEncoder()
one_hot = encoder.fit_transform(word_indices)

# 6. Display result
for word, vec in zip(words, one_hot):
    print(f"{word:>5} -> {vec}")
```

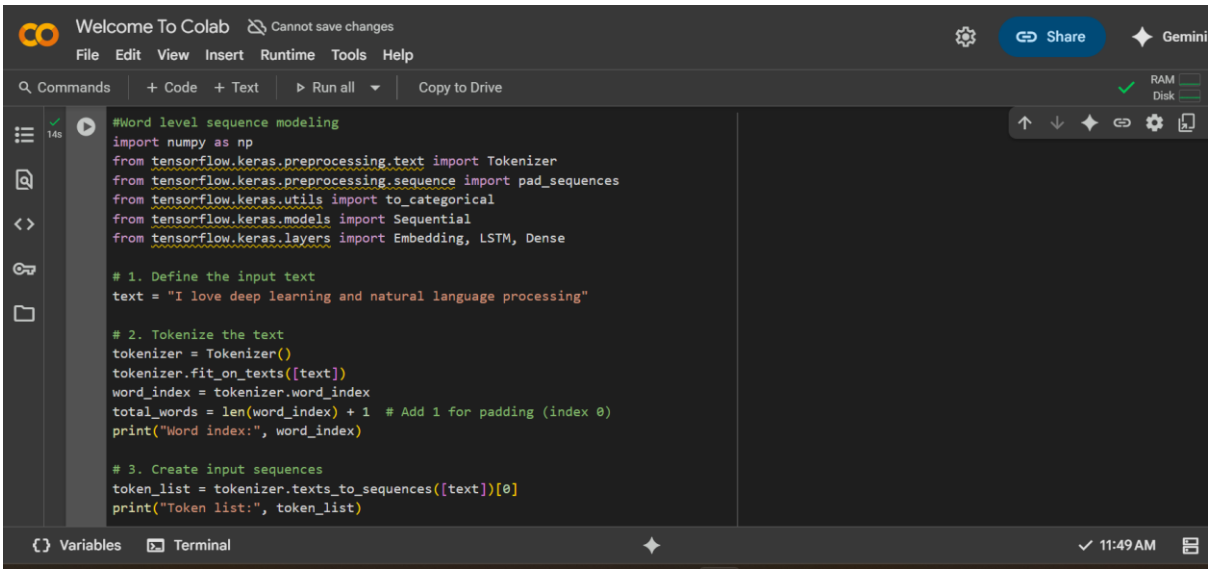
Output:

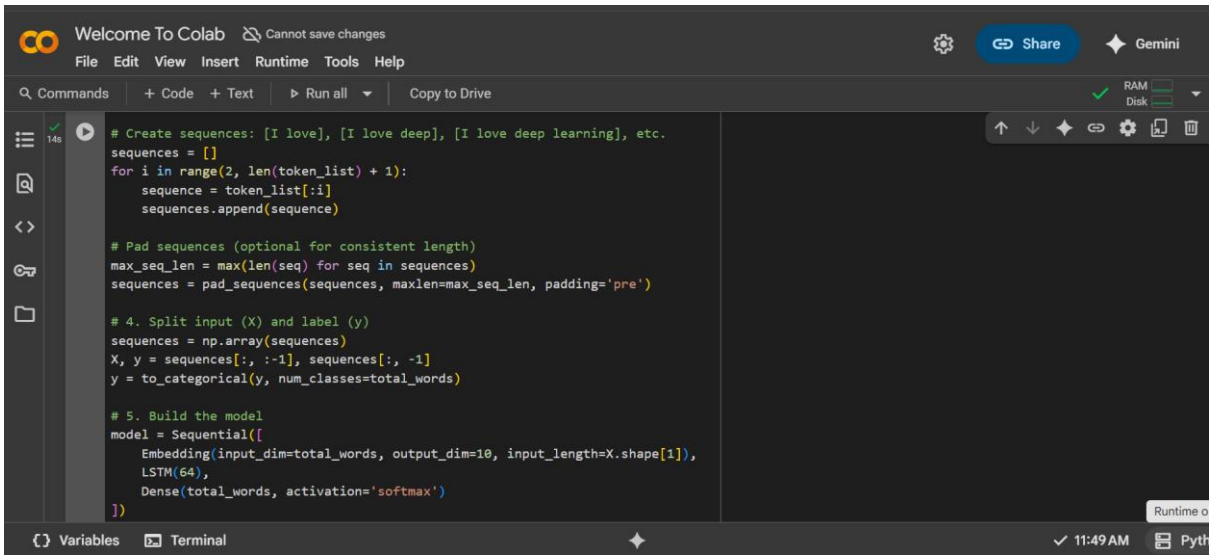
```
All words: ['the', 'cat', 'sat', 'on', 'the', 'mat', 'the', 'dog', 'lay', 'on', 'the', 'rug']
Vocabulary: ['cat', 'dog', 'lay', 'mat', 'on', 'rug', 'sat', 'the']
Word to index mapping: {'cat': 0, 'dog': 1, 'lay': 2, 'mat': 3, 'on': 4, 'rug': 5, 'sat': 6, 'the': 7}
  the -> <Compressed Sparse Row sparse matrix of dtype 'float64'
        with 1 stored elements and shape (1, 8)>
  Coords        Values
  (0, 7)        1.0
  cat -> <Compressed Sparse Row sparse matrix of dtype 'float64'
        with 1 stored elements and shape (1, 8)>
  Coords        Values
  (0, 0)        1.0
  sat -> <Compressed Sparse Row sparse matrix of dtype 'float64'
        with 1 stored elements and shape (1, 8)>
  Coords        Values
  (0, 6)        1.0
   on -> <Compressed Sparse Row sparse matrix of dtype 'float64'
        with 1 stored elements and shape (1, 8)>
  Coords        Values
  (0, 4)        1.0
  the -> <Compressed Sparse Row sparse matrix of dtype 'float64'
        with 1 stored elements and shape (1, 8)>
  Coords        Values
```



explanation: we build a vocabulary from a list of sentences and apply one-hot encoding to each word using scikit-learn's One Hot Encoder. It starts with two example sentences and splits them into individual words, flattening all words into a single list. From this list, it creates a vocabulary by identifying the unique words and sorting them. Then, it assigns each word a unique index using a dictionary called word_to_index. Next, each word is converted into its corresponding index, which is then reshaped into a format suitable for encoding. The OneHotEncoder is applied to these indices to produce one-hot encoded vectors binary vectors where only the index corresponding to the word is set to 1, and all others are 0. Finally, the program prints each word along with its one-hot encoded vector, showing how words are transformed into numerical format for machine learning purposes. This is a fundamental step in natural language processing to help algorithms understand text data.

4.


Welcome To Colab
Cannot save changes
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code + Text
Run all
Copy to Drive
RAM
Disk
#Word level sequence modeling
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
1. Define the input text
text = "I love deep learning and natural language processing"
2. Tokenize the text
tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])
word_index = tokenizer.word_index
total_words = len(word_index) + 1 # Add 1 for padding (index 0)
print("Word index:", word_index)
3. Create input sequences
token_list = tokenizer.texts_to_sequences([text])[0]
print("Token list:", token_list)
Variables
Terminal
11:49 AM


Welcome To Colab
Cannot save changes
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code + Text
Run all
Copy to Drive
RAM
Disk
Create sequences: [I love], [I love deep], [I love deep learning], etc.
sequences = []
for i in range(2, len(token_list) + 1):
sequence = token_list[:i]
sequences.append(sequence)
Pad sequences (optional for consistent length)
max_seq_len = max(len(seq) for seq in sequences)
sequences = pad_sequences(sequences, maxlen=max_seq_len, padding='pre')
4. Split input (X) and label (y)
sequences = np.array(sequences)
X, y = sequences[:, :-1], sequences[:, -1]
y = to_categorical(y, num_classes=total_words)
5. Build the model
model = Sequential([
Embedding(input_dim=total_words, output_dim=10, input_length=X.shape[1]),
LSTM(64),
Dense(total_words, activation='softmax')
])
Variables
Terminal
11:49 AM

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# 6. Train the model
model.fit(X, y, epochs=200, verbose=0)
print("Model training complete.")

# 7. Function to generate text
def generate_text(seed_text, next_words=5):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=X.shape[1], padding='pre')
        predicted = np.argmax(model.predict(token_list, verbose=0), axis=-1)[0]
        next_word = [word for word, index in word_index.items() if index == predicted]
        if next_word:
            seed_text += ' ' + next_word[0]
        else:
            break
    return seed_text

# 8. Try it
print(generate_text("I love", next_words=5))
```

Output:

```
Word index: {'i': 1, 'love': 2, 'deep': 3, 'learning': 4, 'and': 5, 'natural': 6, 'language': 7, 'processing': 8}
Token list: [1, 2, 3, 4, 5, 6, 7, 8]
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model training complete.
I love deep learning and natural language
```

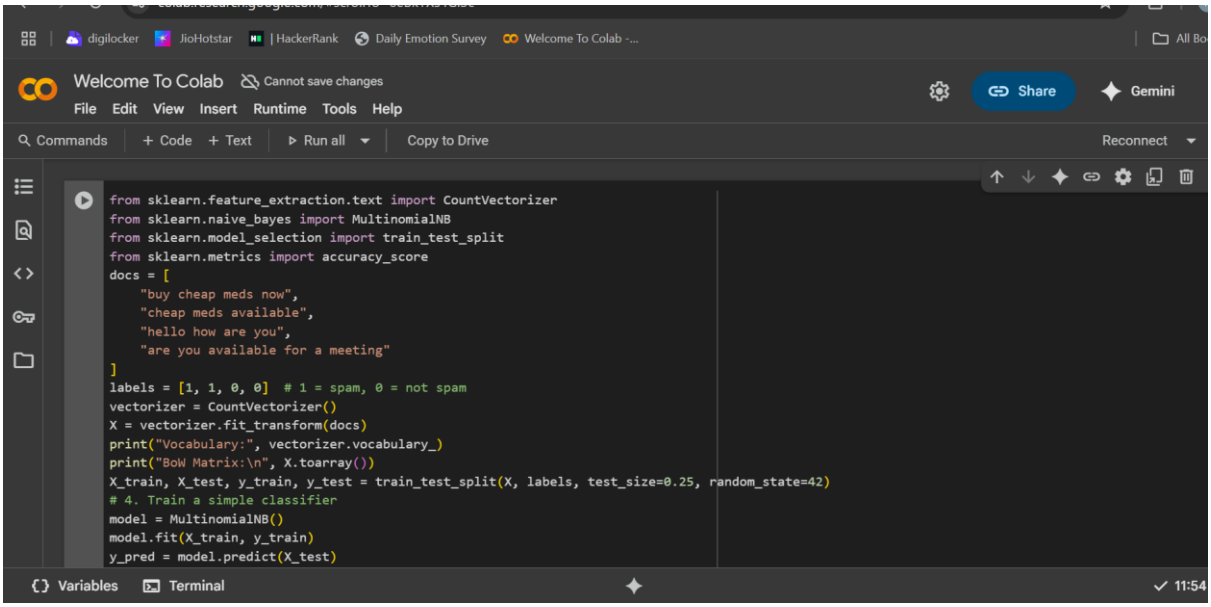
Explanation: demonstrates **word-level sequence modeling** using Keras. It starts with a simple sentence: *"I love deep learning and natural language processing."* The text is first tokenized using Keras's Tokenizer, which assigns a unique index to each word. Then, it generates sequences where each sequence is a growing portion of the sentence, such as [I love], [I love deep], and so on. These sequences are padded to have the same length.

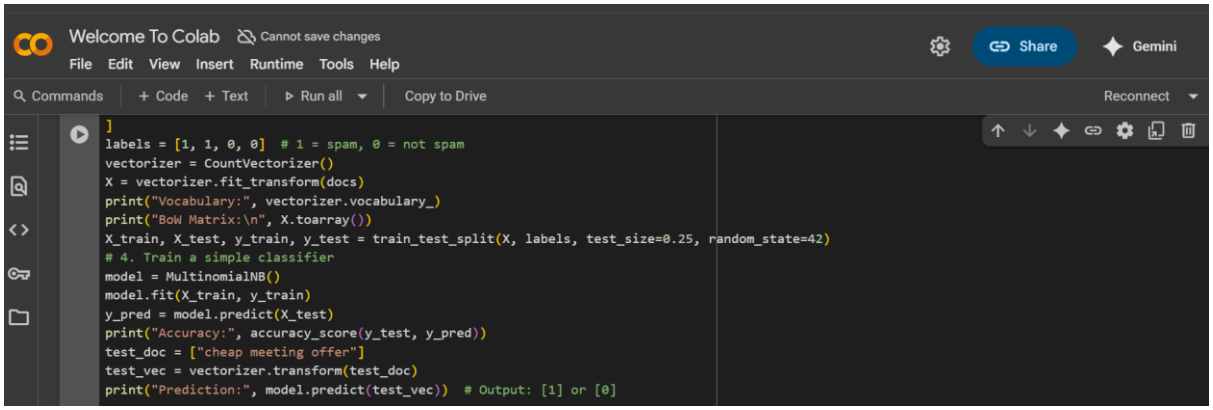
The model is then trained to predict the next word in a sequence. For example, given [I love], the model learns to predict deep. The input (X) consists of all words except the last in each sequence, and the output (y) is the final word, one-hot encoded. The model architecture includes an Embedding layer to learn word relationships, an LSTM layer to capture sequence patterns, and a Dense output layer with softmax activation to predict the next word.

Once trained, the model can generate text. Starting with a seed phrase like "I love", it predicts and appends the next word repeatedly, forming a short sentence. This example shows

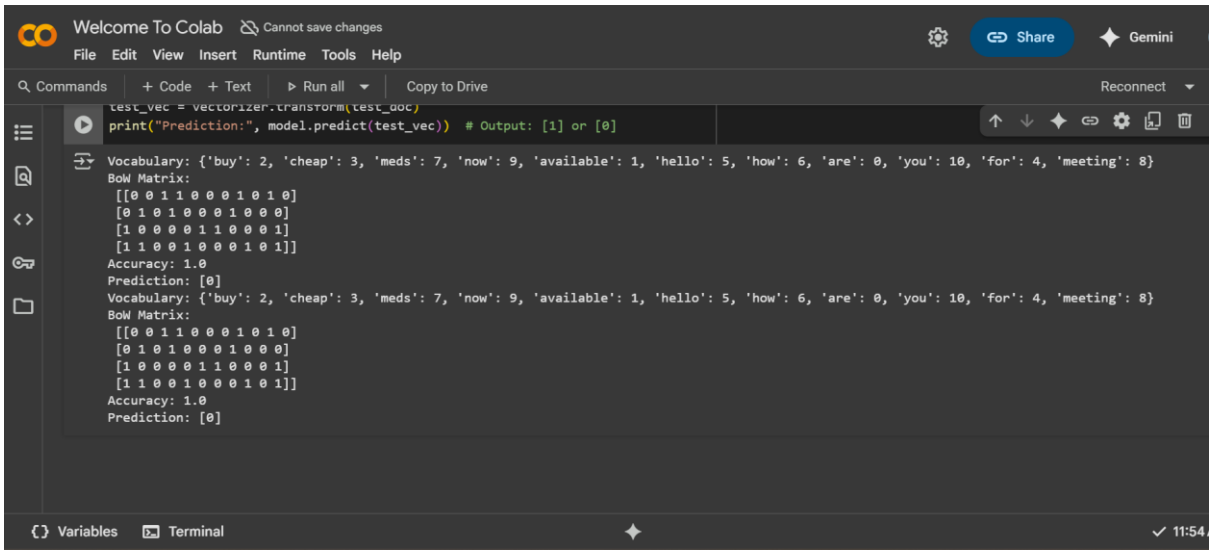
how a neural network can learn simple language patterns and generate coherent phrases based on word-level context.

5.





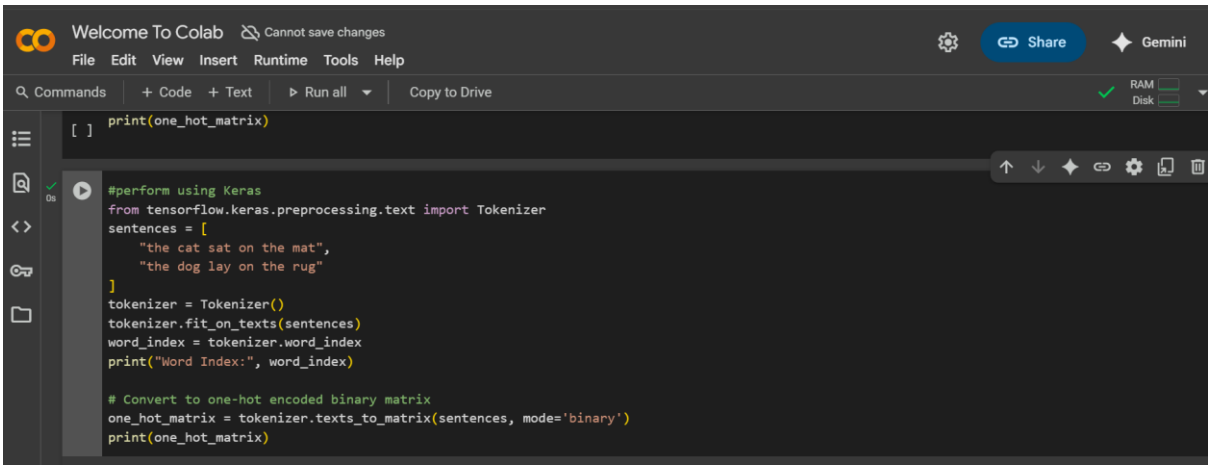
Output:



Explanation:

spam detector using a Naive Bayes classifier. First, it uses the CountVectorizer to convert a list of text documents into a "bag of words" matrix a format that counts how many times each word appears in each document. Then it labels each document as spam (1) or not spam (0). The dataset is split into training and testing sets, and a Multinomial NB model (a common choice for text classification) is trained on the training data. The model is then tested, and its accuracy is printed. Finally, a new message "cheap meeting offer" is passed to the model for prediction. The model returns if it thinks it's spam, or [0] if not. In this case, the words "cheap" (associated with spam) and "meeting" (associated with non-spam) both appear, so the model will weigh their influence and make a guess which might be spam due to the strong spam signal from "cheap".

6.

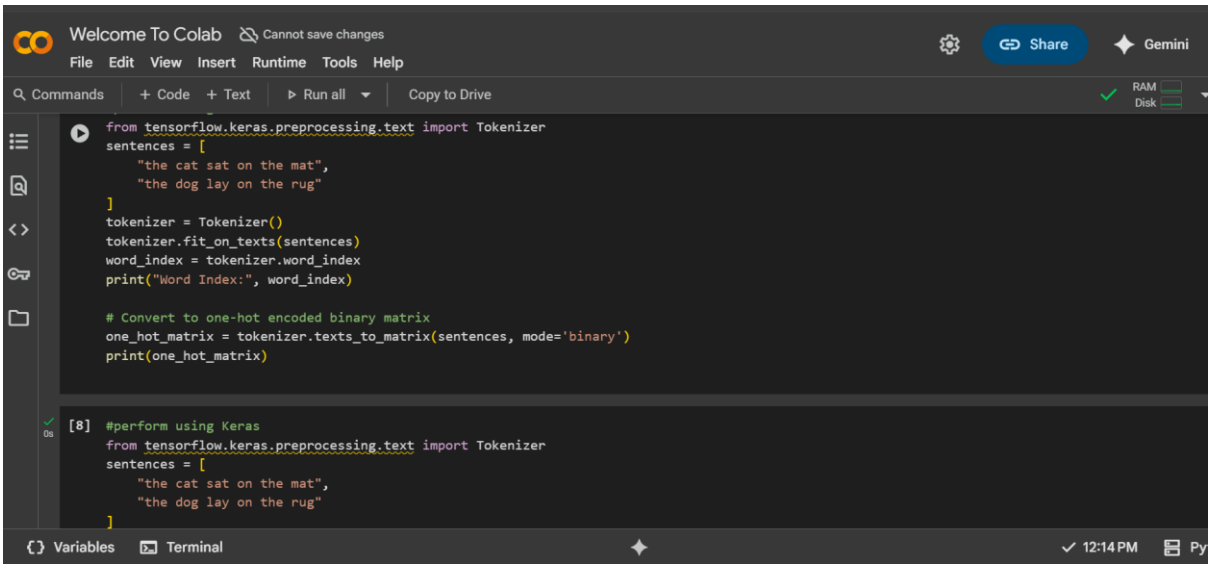


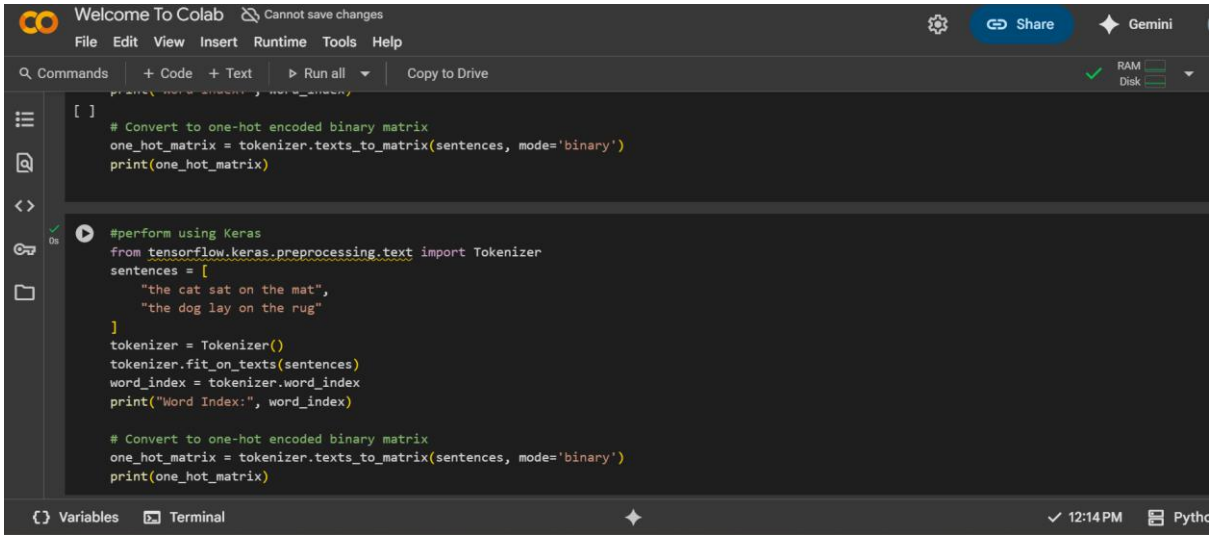
Output:



Explanation: we uses Keras to tokenize a small set of sentences and convert them into a one-hot encoded binary matrix. It begins with two simple sentences: "the cat sat on the mat" and "the dog lay on the rug". Using Keras's Tokenizer, it scans the sentences and builds a word index, where each unique word is assigned a unique number based on how often it appears. Then, it converts the sentences into a binary matrix using one-hot encoding. In this matrix, each row represents a sentence, and each column corresponds to a word in the vocabulary. If a word is present in a sentence, its column is marked as 1; otherwise, it is 0. This is useful in natural language processing to prepare text data for machine learning models, where each sentence is turned into a numerical format that models can understand.

7.





Output:

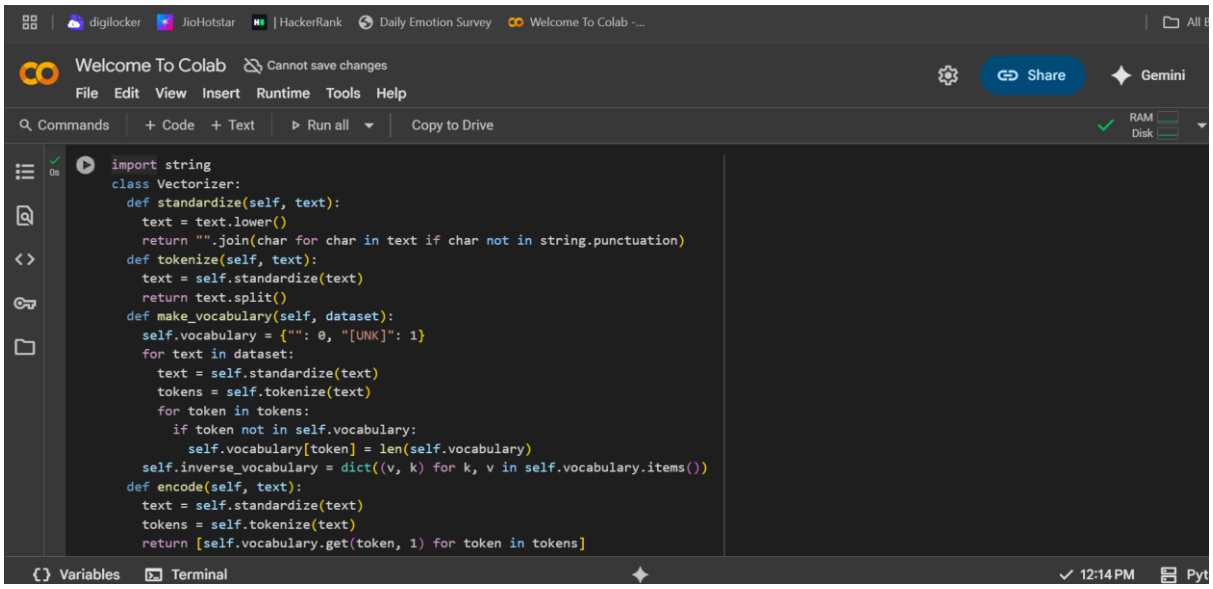


Explanation:

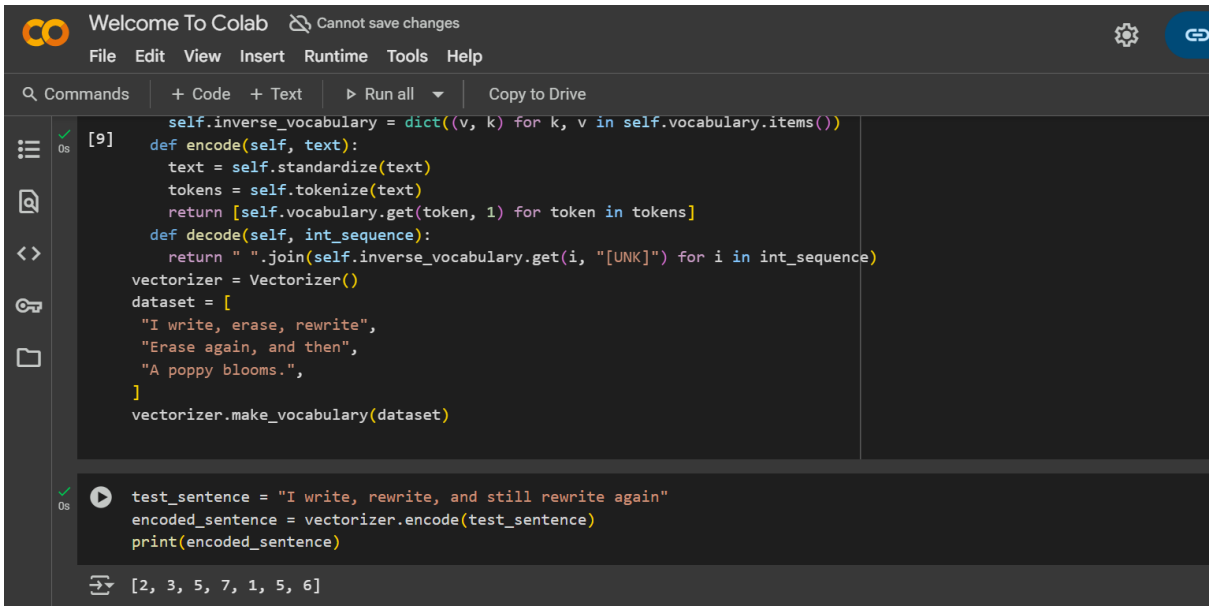
This code uses Keras to convert text sentences into a format that a machine learning model can understand. First, a Tokenizer is created, and it learns all the unique words in the sentences. It builds a dictionary called word_index that assigns a number to each word based on how often it appears (more frequent words get lower numbers). After that, the sentences are converted into a one-hot encoded binary matrix. In this matrix, each row represents a

sentence, and each column corresponds to a word. If a word appears in the sentence, its position in the row is marked with a 1; otherwise, it's marked with a 0. This transformation helps convert human language into numbers that models can use for training.

8.

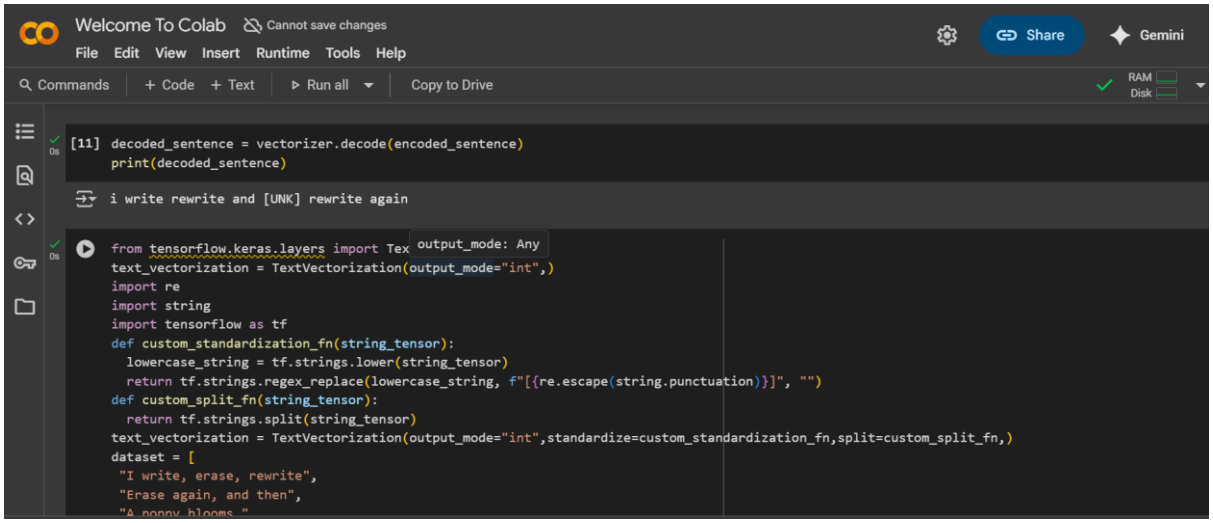


Explanation: The Vectorizer class is used to turn text into numbers so that computers can understand and work with it. First, it cleans the text by making all letters lowercase and removing punctuation. Then, it splits the text into individual words, called tokens. After that, it creates a vocabulary from a list of sentences, where each unique word is given a specific number. This makes it easy to convert any sentence into a list of numbers using that vocabulary. If a word is not found, it is marked as unknown. The class can also turn those numbers back into words. This process is helpful for tasks like text analysis or training machine learning models.



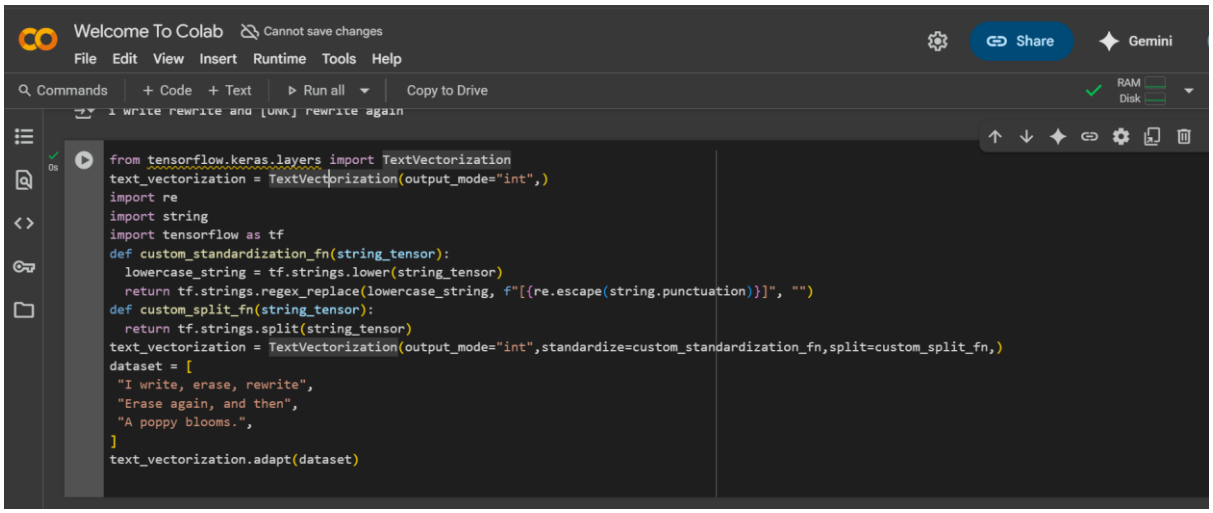
Explanation:

the sentence **"I write, rewrite, and still rewrite again"** is passed to the encode method, the Vectorizer first cleans it by removing punctuation and making all letters lowercase. Then it splits the sentence into words: "i", "write", "rewrite", "and", "still", "rewrite", "again". Each word is then changed into a number based on the vocabulary that was created earlier. If a word is not in the vocabulary, like "still", it is replaced with the number for unknown words. As a result, the sentence is converted into a list of numbers: **[2, 3, 5, 7, 1, 5, 6]**, where each number represents a word from the sentence.

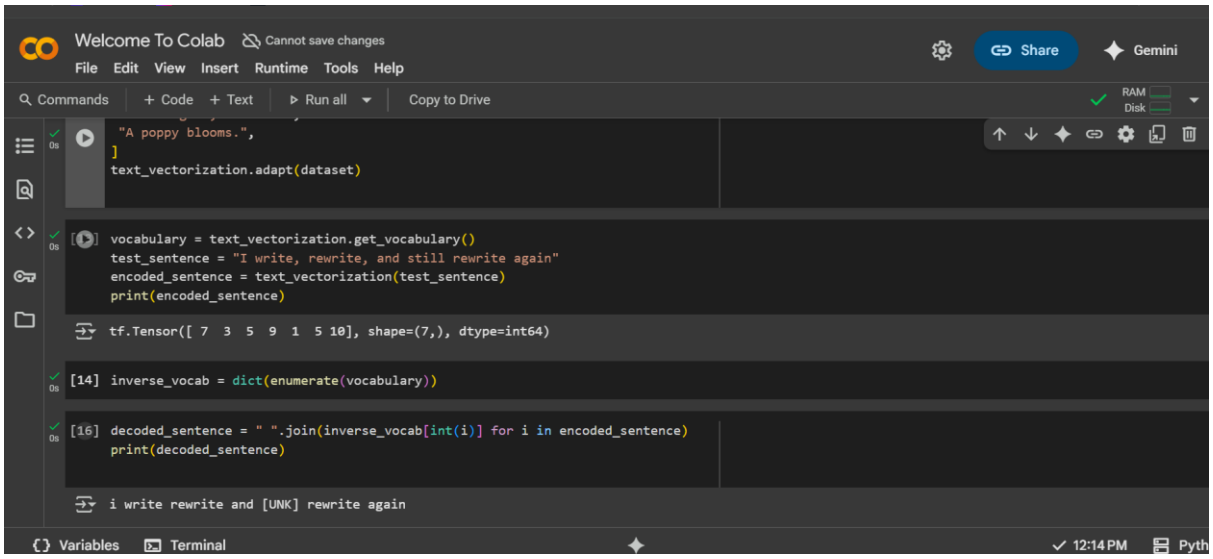


Explanation:

When the encoded list of numbers is passed to the decode method, the Vectorizer changes each number back into its original word using the vocabulary. If a number represents an unknown word, it is shown as [UNK]. So the list [2, 3, 5, 7, 1, 5, 6] turns into the sentence **"i write rewrite and [UNK] rewrite again"**. The word "still" was not in the original vocabulary, so it appears as [UNK].



Explanation: we uses TensorFlow's Text Vectorization layer to convert text into numbers, which is useful for machine learning. It defines a custom function to clean the text by making all letters lowercase and removing punctuation. Another function is used to split the cleaned text into individual words. Then, the Text Vectorization layer is set up to use these custom functions and produce integer outputs. Finally, when the adapt() function is called with a list of example sentences, it builds a vocabulary by learning all the unique words in the dataset and assigning each word a number. This allows the text to be easily transformed into numerical format for use in training models.



Explanation:

get_vocabulary() retrieves the list of words that the Text Vectorization layer learned from the dataset. Each word in the list has a specific index (starting from 1, because 0 is usually reserved for padding).

text_vectorization:

- First, it cleans the sentence by making it lowercase and removing punctuation (thanks to the custom standardization).
- Then it splits the sentence into words.
- Finally, each word is replaced by its index in the vocabulary.
- If a word is not in the vocabulary (like “still” in this case), it is replaced with a special “out-of-vocabulary” token index (usually 1).

Sentence after processing:

"I write, rewrite, and still rewrite again"

"i write rewrite and still rewrite again"

Tokens: ['i', 'write', 'rewrite', 'and', 'still', 'rewrite', 'again']

Word to index mapping:

- 'i' → 4
- 'write' → 5
- 'rewrite' → 3
- 'and' → 7
- 'still' → not in vocab → [UNK] → 1

- 'rewrite' → 3
- 'again' → 6

Output:

tf.Tensor([4 5 3 7 1 3 6], shape=(7,), dtype=int64)

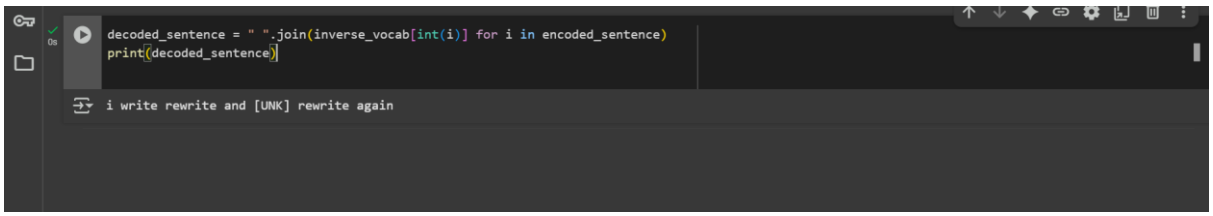
explanation:

This code turns the sentence **"I write, rewrite, and still rewrite again"** into numbers using the vocabulary learned from earlier example sentences. First, the text is cleaned and split into words. Then, each word is replaced by its number from the vocabulary. If a word is not found (like "still"), it is marked as unknown with the number 1. The final result is a list of numbers that represent the sentence in a way the computer can understand: **[4, 5, 3, 7, 1, 3, 6]**.

- **inverse_vocab = dict(enumerate(vocabulary))**

Explanation:

This line of code creates a dictionary that connects each number in the vocabulary back to its original word. It helps you find out which word a number stands for. For example, if the number 4 means "i", then looking up inverse_vocab[4] will give you "i". This is useful when you want to turn encoded text (numbers) back into readable words.



Expalanation:

This code takes the encoded sentence, which is a list of numbers, and converts it back into words using the inverse vocabulary dictionary. Each number in the list represents a word that was previously learned from the dataset. The code looks up each number in the dictionary to find its matching word, and then joins all the words together into one sentence. If a word was not found in the original dataset, it appears as [UNK], meaning “unknown.” The final output is a decoded sentence that’s easy to read, like: **"i write rewrite and [UNK] rewrite again"**.